

FANUC Robot **series**

NC Program Conversion

User's Guide

Version 7.70 and later

MARUCNCPR06121E REV D

This publication contains proprietary information of FANUC Robotics America Corporation furnished for customer use only. No other uses are authorized without the express written permission of FANUC Robotics America Corporation.

**FANUC Robotics America Corporation
3900 W. Hamlin Road
Rochester Hills, Michigan 48309-3253**

Copyrights and Trademarks

This new publication contains proprietary information of FANUC America Corporation furnished for customer use only. No other uses are authorized without the express written permission of FANUC America Corporation.

The descriptions and specifications contained in this manual were in effect at the time this manual was approved for printing. FANUC America Corporation, hereinafter referred to as FANUC, reserves the right to discontinue models at any time or to change specifications or design without notice and without incurring obligations.

FANUC manuals present descriptions, specifications, drawings, schematics, bills of material, parts, connections and/or procedures for installing, disassembling, connecting, operating and programming FANUC products and/or systems. Such systems consist of robots, extended axes, robot controllers, application software, the KAREL® programming language, INSIGHT® vision equipment, and special tools.

FANUC recommends that only persons who have been trained in one or more approved FANUC Training Course(s) be permitted to install, operate, use, perform procedures on, repair, and/or maintain FANUC products and/or systems and their respective components. Approved training necessitates that the courses selected be relevant to the type of system installed and application performed at the customer site.



WARNING

This equipment generates, uses, and can radiate radiofrequency energy and if not installed and used in accordance with the instruction manual, may cause interference to radio communications. As temporarily permitted by regulation, it has not been tested for compliance with the limits for Class A computing devices pursuant to Subpart J and Part 15 of FCC Rules, which are designed to provide reasonable protection against such interference. Operation of the equipment in a residential area is likely to cause interference, in which case the user, at his own expense, will be required to take whatever measure may be required to correct the interference.

FANUC conducts courses on its systems and products on a regularly scheduled basis at the company's world headquarters in Rochester Hills, Michigan. For additional information contact

FANUC America Corporation
Training Department
3900 W. Hamlin Road
Rochester Hills, Michigan 48309-3253
www.fanucamerica.com

For customer assistance, including Technical Support, Service, Parts & Part Repair, and Marketing Requests, contact the Customer Resource Center, 24 hours a day, at 888-FANUC-US (888-326-8287).

Send your comments and suggestions about this manual to:
product.documentation@fanucamerica.com

**Copyright © 2018 by FANUC America Corporation
All Rights Reserved**

The information illustrated or contained herein is not to be reproduced, copied, downloaded, translated into another language, published in any physical or electronic format, including internet, or transmitted in whole or in part in any way without the prior written consent of FANUC America Corporation.

AccuStat®, ArcTool®, iRVision®, KAREL®, PaintTool®, PalletTool®, SOCKETS®, SpotTool®, SpotWorks®, and TorchMate® are Registered Trademarks of FANUC.

FANUC reserves all proprietary rights, including but not limited to trademark and trade name rights, in the following names:

AccuAir™, AccuCal™, AccuChop™, AccuFlow™, AccuPath™, AccuSeal™, ARC Mate™, ARC Mate Sr.™, ARC Mate System 1™, ARC Mate System 2™, ARC Mate System 3™, ARC Mate System 4™, ARC Mate System 5™, ARCWorks Pro™, AssistTool™, AutoNormal™, AutoTCP™, BellTool™, BODYWorks™, Cal Mate™, Cell Finder™, Center Finder™, Clean Wall™, DualARM™, iRProgrammer™, LR Tool™, MIG Eye™, MotionParts™, MultiARM™, NoBots™, Paint Stick™, PaintPro™, PaintTool 100™, PAINTWorks™, PAINTWorks II™, PAINTWorks III™, PalletMate™, PalletMate PC™, PalletTool PC™, PayloadID™, RecipTool™, RemovalTool™, Robo Chop™, Robo Spray™, S-420i™, S-430i™, ShapeGen™, SoftFloat™, SOFT PARTS™, SpotTool+™, SR Mate™, SR ShotTool™, SureWeld™, SYSTEM R-J2 Controller™, SYSTEM R-J3 Controller™, SYSTEM R-J3iB Controller™, SYSTEM R-J3iC Controller™, SYSTEM R-30iA Controller™, SYSTEM R-30iA Mate Controller™, SYSTEM R-30iB Controller™, SYSTEM R-30iB Mate Controller™, SYSTEM R-30iB Plus Controller™, SYSTEM R-30iB Mate Plus Controller™, TCP Mate™, TorchMate™, TripleARM™, TurboMove™, visLOC™, visPRO-3D™, visTRAC™, WebServer™, WebTP™, and YagTool™.

©FANUC CORPORATION 2018

- No part of this manual may be reproduced in any form.
- All specifications and designs are subject to change without notice.

Patents

One or more of the following U.S. patents might be related to the FANUC products described in this manual.

FANUC America Corporation Patent List

4,630,567 4,639,878 4,707,647 4,708,175 4,708,580 4,942,539 4,984,745
5,238,029 5,239,739 5,272,805 5,293,107 5,293,911 5,331,264 5,367,944
5,373,221 5,421,218 5,434,489 5,644,898 5,670,202 5,696,687 5,737,218
5,823,389 5,853,027 5,887,800 5,941,679 5,959,425 5,987,726 6,059,092
6,064,168 6,070,109 6,086,294 6,122,062 6,147,323 6,204,620 6,243,621
6,253,799 6,285,920 6,313,595 6,325,302 6,345,818 6,356,807 6,360,143
6,378,190 6,385,508 6,425,177 6,477,913 6,490,369 6,518,980 6,540,104
6,541,757 6,560,513 6,569,258 6,612,449 6,703,079 6,705,361 6,726,773
6,768,078 6,845,295 6,945,483 7,149,606 7,149,606 7,211,978 7,266,422
7,399,363

FANUC CORPORATION Patent List

4,571,694 4,626,756 4,700,118 4,706,001 4,728,872 4,732,526 4,742,207
4,835,362 4,894,596 4,899,095 4,920,248 4,931,617 4,934,504 4,956,594
4,967,125 4,969,109 4,970,370 4,970,448 4,979,127 5,004,968 5,006,035
5,008,834 5,063,281 5,066,847 5,066,902 5,093,552 5,107,716 5,111,019
5,130,515 5,136,223 5,151,608 5,170,109 5,189,351 5,267,483 5,274,360
5,292,066 5,300,868 5,304,906 5,313,563 5,319,443 5,325,467 5,327,057
5,329,469 5,333,242 5,337,148 5,371,452 5,375,480 5,418,441 5,432,316
5,440,213 5,442,155 5,444,612 5,449,875 5,451,850 5,461,478 5,463,297
5,467,003 5,471,312 5,479,078 5,485,389 5,485,552 5,486,679 5,489,758
5,493,192 5,504,766 5,511,007 5,520,062 5,528,013 5,532,924 5,548,194
5,552,687 5,558,196 5,561,742 5,570,187 5,570,190 5,572,103 5,581,167
5,582,750 5,587,635 5,600,759 5,608,299 5,608,618 5,624,588 5,630,955
5,637,969 5,639,204 5,641,415 5,650,078 5,658,121 5,668,628 5,687,295
5,691,615 5,698,121 5,708,342 5,715,375 5,719,479 5,727,132 5,742,138
5,742,144 5,748,854 5,749,058 5,760,560 5,773,950 5,783,922 5,799,135
5,812,408 5,841,257 5,845,053 5,872,894 5,887,122 5,911,892 5,912,540
5,920,678 5,937,143 5,980,082 5,983,744 5,987,591 5,988,850 6,023,044
6,032,086 6,040,554 6,059,169 6,088,628 6,097,169 6,114,824 6,124,693
6,140,788 6,141,863 6,157,155 6,160,324 6,163,124 6,177,650 6,180,898
6,181,096 6,188,194 6,208,105 6,212,444 6,219,583 6,226,181 6,236,011
6,236,896 6,250,174 6,278,902 6,279,413 6,285,921 6,298,283 6,321,139
6,324,443 6,328,523 6,330,493 6,340,875 6,356,671 6,377,869 6,382,012
6,384,371 6,396,030 6,414,711 6,424,883 6,431,018 6,434,448 6,445,979
6,459,958 6,463,358 6,484,067 6,486,629 6,507,165 6,654,666 6,665,588
6,680,461 6,696,810 6,728,417 6,763,284 6,772,493 6,845,296 6,853,881
6,888,089 6,898,486 6,917,837 6,928,337 6,965,091 6,970,802 7,038,165
7,069,808 7,084,900 7,092,791 7,133,747 7,143,100 7,149,602 7,131,848
7,161,321 7,171,041 7,174,234 7,173,213 7,177,722 7,177,439 7,181,294
7,181,313 7,280,687 7,283,661 7,291,806 7,299,713 7,315,650 7,324,873
7,328,083 7,330,777 7,333,879 7,355,725 7,359,817 7,373,220 7,376,488
7,386,367 7,464,623 7,447,615 7,445,260 7,474,939 7,486,816 7,495,192
7,501,778 7,502,504 7,508,155 7,512,459 7,525,273 7,526,121

Conventions



WARNING

Information appearing under the "WARNING" caption concerns the protection of personnel. It is boxed and bolded to set it apart from the surrounding text.



CAUTION

Information appearing under the "CAUTION" caption concerns the protection of equipment, software, and data. It is boxed and bolded to set it apart from the surrounding text.

Note Information appearing next to NOTE concerns related information or useful hints.

Safety

FANUC America Corporation is not and does not represent itself as an expert in safety systems, safety equipment, or the specific safety aspects of your company and/or its work force. It is the responsibility of the owner, employer, or user to take all necessary steps to guarantee the safety of all personnel in the workplace.

The appropriate level of safety for your application and installation can be best determined by safety system professionals. FANUC America Corporation therefore, recommends that each customer consult with such professionals in order to provide a workplace that allows for the safe application, use, and operation of FANUC America Corporation systems.

According to the industry standard ANSI/RIA R15-06, the owner or user is advised to consult the standards to ensure compliance with its requests for Robotics System design, usability, operation, maintenance, and service. Additionally, as the owner, employer, or user of a robotic system, it is your responsibility to arrange for the training of the operator of a robot system to recognize and respond to known hazards associated with your robotic system and to be aware of the recommended operating procedures for your particular application and robot installation.

Ensure that the robot being used is appropriate for the application. Robots used in classified (hazardous) locations must be certified for this use.

FANUC America Corporation therefore, recommends that all personnel who intend to operate, program, repair, or otherwise use the robotics system be trained in an approved FANUC America Corporation training course and become familiar with the proper operation of the system. Persons responsible for programming the system—including the design, implementation, and debugging of application programs—must be familiar with the recommended programming procedures for your application and robot installation.

The following guidelines are provided to emphasize the importance of safety in the workplace.

CONSIDERING SAFETY FOR YOUR ROBOT INSTALLATION

Safety is essential whenever robots are used. Keep in mind the following factors with regard to safety:

- The safety of people and equipment
- Use of safety enhancing devices
- Techniques for safe teaching and manual operation of the robot(s)
- Techniques for safe automatic operation of the robot(s)
- Regular scheduled inspection of the robot and workcell
- Proper maintenance of the robot

Keeping People Safe

The safety of people is always of primary importance in any situation. When applying safety measures to your robotic system, consider the following:

- External devices
- Robot(s)
- Tooling
- Workpiece

Using Safety Enhancing Devices

Always give appropriate attention to the work area that surrounds the robot. The safety of the work area can be enhanced by the installation of some or all of the following devices:

- Safety fences, barriers, or chains
- Light curtains
- Interlocks
- Pressure mats
- Floor markings
- Warning lights
- Mechanical stops
- EMERGENCY STOP buttons
- DEADMAN switches

Setting Up a Safe Workcell

A safe workcell is essential to protect people and equipment. Observe the following guidelines to ensure that the workcell is set up safely. These suggestions are intended to supplement and not replace existing federal, state, and local laws, regulations, and guidelines that pertain to safety.

- Sponsor your personnel for training in approved FANUC America Corporation training course(s) related to your application. Never permit untrained personnel to operate the robots.
- Install a lockout device that uses an access code to prevent unauthorized persons from operating the robot.
- Use anti-tie-down logic to prevent the operator from bypassing safety measures.
- Arrange the workcell so the operator faces the workcell and can see what is going on inside the cell.
- Clearly identify the work envelope of each robot in the system with floor markings, signs, and special barriers. The work envelope is the area defined by the maximum motion range of the robot, including any tooling attached to the wrist flange that extend this range.

- Position all controllers outside the robot work envelope.
- Never rely on software or firmware based controllers as the primary safety element unless they comply with applicable current robot safety standards.
- Mount an adequate number of EMERGENCY STOP buttons or switches within easy reach of the operator and at critical points inside and around the outside of the workcell.
- Install flashing lights and/or audible warning devices that activate whenever the robot is operating, that is, whenever power is applied to the servo drive system. Audible warning devices shall exceed the ambient noise level at the end-use application.
- Wherever possible, install safety fences to protect against unauthorized entry by personnel into the work envelope.
- Install special guarding that prevents the operator from reaching into restricted areas of the work envelope.
- Use interlocks.
- Use presence or proximity sensing devices such as light curtains, mats, and capacitance and vision systems to enhance safety.
- Periodically check the safety joints or safety clutches that can be optionally installed between the robot wrist flange and tooling. If the tooling strikes an object, these devices dislodge, remove power from the system, and help to minimize damage to the tooling and robot.
- Make sure all external devices are properly filtered, grounded, shielded, and suppressed to prevent hazardous motion due to the effects of electro-magnetic interference (EMI), radio frequency interference (RFI), and electro-static discharge (ESD).
- Make provisions for power lockout/tagout at the controller.
- Eliminate *pinch points*. Pinch points are areas where personnel could get trapped between a moving robot and other equipment.
- Provide enough room inside the workcell to permit personnel to teach the robot and perform maintenance safely.
- Program the robot to load and unload material safely.
- If high voltage electrostatics are present, be sure to provide appropriate interlocks, warning, and beacons.
- If materials are being applied at dangerously high pressure, provide electrical interlocks for lockout of material flow and pressure.

Staying Safe While Teaching or Manually Operating the Robot

Advise all personnel who must teach the robot or otherwise manually operate the robot to observe the following rules:

- Never wear watches, rings, neckties, scarves, or loose clothing that could get caught in moving machinery.
- Know whether or not you are using an intrinsically safe teach pendant if you are working in a hazardous environment.

- Before teaching, visually inspect the robot and work envelope to make sure that no potentially hazardous conditions exist. The work envelope is the area defined by the maximum motion range of the robot. These include tooling attached to the wrist flange that extends this range.
- The area near the robot must be clean and free of oil, water, or debris. Immediately report unsafe working conditions to the supervisor or safety department.
- FANUC America Corporation recommends that no one enter the work envelope of a robot that is on, except for robot teaching operations. However, if you must enter the work envelope, be sure all safeguards are in place, check the teach pendant DEADMAN switch for proper operation, and place the robot in teach mode. Take the teach pendant with you, turn it on, and be prepared to release the DEADMAN switch. Only the person with the teach pendant should be in the work envelope.

 WARNING

Never bypass, strap, or otherwise deactivate a safety device, such as a limit switch, for any operational convenience. Deactivating a safety device is known to have resulted in serious injury and death.

- Know the path that can be used to escape from a moving robot; make sure the escape path is never blocked.
- Isolate the robot from all remote control signals that can cause motion while data is being taught.
- Test any program being run for the first time in the following manner:

 WARNING

Stay outside the robot work envelope whenever a program is being run. Failure to do so can result in injury.

- Using a low motion speed, single step the program for at least one full cycle.
- Using a low motion speed, test run the program continuously for at least one full cycle.
- Using the programmed speed, test run the program continuously for at least one full cycle.
- Make sure all personnel are outside the work envelope before running production.

Staying Safe During Automatic Operation

Advise all personnel who operate the robot during production to observe the following rules:

- Make sure all safety provisions are present and active.

- Know the entire workcell area. The workcell includes the robot and its work envelope, plus the area occupied by all external devices and other equipment with which the robot interacts.
- Understand the complete task the robot is programmed to perform before initiating automatic operation.
- Make sure all personnel are outside the work envelope before operating the robot.
- Never enter or allow others to enter the work envelope during automatic operation of the robot.
- Know the location and status of all switches, sensors, and control signals that could cause the robot to move.
- Know where the EMERGENCY STOP buttons are located on both the robot control and external control devices. Be prepared to press these buttons in an emergency.
- Never assume that a program is complete if the robot is not moving. The robot could be waiting for an input signal that will permit it to continue its activity.
- If the robot is running in a pattern, do not assume it will continue to run in the same pattern.
- Never try to stop the robot, or break its motion, with your body. The only way to stop robot motion immediately is to press an EMERGENCY STOP button located on the controller panel, teach pendant, or emergency stop stations around the workcell.

Staying Safe During Inspection

When inspecting the robot, be sure to

- Turn off power at the controller.
- Lock out and tag out the power source at the controller according to the policies of your plant.
- Turn off the compressed air source and relieve the air pressure.
- If robot motion is not needed for inspecting the electrical circuits, press the EMERGENCY STOP button on the operator panel.
- Never wear watches, rings, neckties, scarves, or loose clothing that could get caught in moving machinery.
- If power is needed to check the robot motion or electrical circuits, be prepared to press the EMERGENCY STOP button, in an emergency.
- Be aware that when you remove a servomotor or brake, the associated robot arm will fall if it is not supported or resting on a hard stop. Support the arm on a solid support before you release the brake.

Staying Safe During Maintenance

When performing maintenance on your robot system, observe the following rules:

- Never enter the work envelope while the robot or a program is in operation.
- Before entering the work envelope, visually inspect the workcell to make sure no potentially hazardous conditions exist.

- Never wear watches, rings, neckties, scarves, or loose clothing that could get caught in moving machinery.
- Consider all or any overlapping work envelopes of adjoining robots when standing in a work envelope.
- Test the teach pendant for proper operation before entering the work envelope.
- If it is necessary for you to enter the robot work envelope while power is turned on, you must be sure that you are in control of the robot. Be sure to take the teach pendant with you, press the DEADMAN switch, and turn the teach pendant on. Be prepared to release the DEADMAN switch to turn off servo power to the robot immediately.
- Whenever possible, perform maintenance with the power turned off. Before you open the controller front panel or enter the work envelope, turn off and lock out the 3-phase power source at the controller.
- Be aware that when you remove a servomotor or brake, the associated robot arm will fall if it is not supported or resting on a hard stop. Support the arm on a solid support before you release the brake.

 WARNING

Lethal voltage is present in the controller WHENEVER IT IS CONNECTED to a power source. Be extremely careful to avoid electrical shock. HIGH VOLTAGE IS PRESENT at the input side whenever the controller is connected to a power source. Turning the disconnect or circuit breaker to the OFF position removes power from the output side of the device only.

- Release or block all stored energy. Before working on the pneumatic system, shut off the system air supply and purge the air lines.
- Isolate the robot from all remote control signals. If maintenance must be done when the power is on, make sure the person inside the work envelope has sole control of the robot. The teach pendant must be held by this person.
- Make sure personnel cannot get trapped between the moving robot and other equipment. Know the path that can be used to escape from a moving robot. Make sure the escape route is never blocked.
- Use blocks, mechanical stops, and pins to prevent hazardous movement by the robot. Make sure that such devices do not create pinch points that could trap personnel.

 WARNING

Do not try to remove any mechanical component from the robot before thoroughly reading and understanding the procedures in the appropriate manual. Doing so can result in serious personal injury and component destruction.

- Be aware that when you remove a servomotor or brake, the associated robot arm will fall if it is not supported or resting on a hard stop. Support the arm on a solid support before you release the brake.
- When replacing or installing components, make sure dirt and debris do not enter the system.
- Use only specified parts for replacement. To avoid fires and damage to parts in the controller, never use nonspecified fuses.
- Before restarting a robot, make sure no one is inside the work envelope; be sure that the robot and all external devices are operating normally.

KEEPING MACHINE TOOLS AND EXTERNAL DEVICES SAFE

Certain programming and mechanical measures are useful in keeping the machine tools and other external devices safe. Some of these measures are outlined below. Make sure you know all associated measures for safe use of such devices.

Programming Safety Precautions

Implement the following programming safety measures to prevent damage to machine tools and other external devices.

- Back-check limit switches in the workcell to make sure they do not fail.
- Implement “failure routines” in programs that will provide appropriate robot actions if an external device or another robot in the workcell fails.
- Use *handshaking* protocol to synchronize robot and external device operations.
- Program the robot to check the condition of all external devices during an operating cycle.

Mechanical Safety Precautions

Implement the following mechanical safety measures to prevent damage to machine tools and other external devices.

- Make sure the workcell is clean and free of oil, water, and debris.
- Use DCS (Dual Check Safety), software limits, limit switches, and mechanical hardstops to prevent undesired movement of the robot into the work area of machine tools and external devices.

KEEPING THE ROBOT SAFE

Observe the following operating and programming guidelines to prevent damage to the robot.

Operating Safety Precautions

The following measures are designed to prevent damage to the robot during operation.

- Use a low override speed to increase your control over the robot when jogging the robot.
- Visualize the movement the robot will make before you press the jog keys on the teach pendant.
- Make sure the work envelope is clean and free of oil, water, or debris.
- Use circuit breakers to guard against electrical overload.

Programming Safety Precautions

The following safety measures are designed to prevent damage to the robot during programming:

- Establish *interference zones* to prevent collisions when two or more robots share a work area.
- Make sure that the program ends with the robot near or at the home position.
- Be aware of signals or other operations that could trigger operation of tooling resulting in personal injury or equipment damage.
- In dispensing applications, be aware of all safety guidelines with respect to the dispensing materials.

NOTE: Any deviation from the methods and safety practices described in this manual must conform to the approved standards of your company. If you have questions, see your supervisor.

Table of Contents

1	OVERVIEW	1-1
1.1	NC PROGRAM SYNTAX	1-1
1.2	FUNCTIONAL SPECIFICATIONS	1-1
1.3	SOFTWARE OPTIONS REQUIRED	1-3
2	OPERATION	2-1
2.1	CONVERSION OPERATION	2-1
2.1.1	Load NC program in FILE menu	2-1
2.1.2	COPY NC program to MD drive	2-2
A	NC PROGRAM SYNTAX	A-1
A.1	O code – Program Name/Number	A-1
A.2	N code – Line numbers	A-1
A.3	G code – Motion modifiers	A-2
A.4	F code – Feed Rate / Speed	A-3
A.5	Position data	A-3
A.6	K code - Config string	A-4
A.7	M code – Miscellaneous codes	A-4
A.7.1	Pre-defined M-codes	A-5
A.7.2	Special M-codes	A-5
A.7.3	Predefined M-codes	A-6
	M00 (compulsory stop)	A-6
	M01 (optional stop)	A-6
	M02 (End of Program)	A-6
	M30 (End of program)	A-7
	M221 (User Frame Selection)	A-7
	M222 (User Tool Selection)	A-7
A.7.4	Other (user defined) M-codes	A-8
	M-codes with no associated T-code	A-8
	M-codes with associated T-code	A-8
B	SYSTEM VARIABLES	B-1

B.1	\$NC_SYTX_TBL.....	B-1
B.2	\$NC_GCOD_TBL : To define G-code numbers with their meaning.....	B-1
B.3	\$NC_MCOD_TBL : To define M-code numbers with their meaning.	B-2
B.4	\$NC_DEFMODAL	B-2
B.5	\$NC_OFST_CMD.....	B-3
B.6	\$NC_TOFS_CMD.....	B-4
B.7	\$NC_VOFS_CMD : Command string to be used for Vision offset command	B-4
B.8	\$NC_DEST_DRV	B-4
C	NC PROGRAM CONVERSION EXAMPLE.....	C-1

1

OVERVIEW

1 OVERVIEW

The NC Program Conversion option (R775) allows you to load a robot program written in the NC program language (also known as G-code programs) into a robot controller, and convert it to the TP programs, thus enabling the robot to execute a user program that is written in the NC program language.

1.1 NC program syntax

As is always the case, an NC program is written specifically for the target machine tool. For example, an NC program for a lathe is different from that for a machining center, and is therefore incompatible. Likewise, an NC program for a robot has to be written specifically for a robot application. This feature is not a general purpose program convertor, but is a feature that can only convert an NC program specifically written for a robot into a robot TP program.

One example is the position representation. The positions must be for the robot mechanical configuration. Typically the positional representation “XYZWPR” is used for robot programs, where each character represents the location and orientation of the robot tool center point (TCP).

The M-codes (miscellaneous codes) are, by definition, provided for each application. With limited configurability, the feature is able to convert a set of pre-defined M-codes into TP program instructions, or CALL to a sub-program that does the processing specific to that M-code. A list of supported M-codes is shown later.

1.2 Functional Specifications

The following is a list of specifications and limitations of this feature.

1. This is a feature used to convert an NC program that is written in ASCII format into a TP program. The file extension for the source program is “.NC”.
2. The output TP program has the same program name as the original NC program (for example, a TP program will have a name: “O0081”; for an NC program: “%O0081”). The source file name has to be the same as the program name, e.g. O0081.NC, etc.
3. There will be line-by-line correspondence between the source NC program and the output TP program with a few exceptions
 - Inline comments are not supported on a TP program.
 - The maximum length of a comment is limited by the TP program specifications, and will be truncated if too long (up to 32 characters for a comment line, up to 16 characters for a comment on a position, etc.).
4. The source NC program must be in compliance with a certain NC program syntax rule, or a conversion error will be posted. For example,
 - Commands are case sensitive. Upper case letters need to be used.
 - All commands (except for the K command) start with a letter followed by one or more numerals. Leading zeros can be omitted.
5. Each line of an NC program needs to begin with an N-code, indicating the program line number.

- The number must increase as it proceeds. No duplicate numbers are allowed
 - The number does not have to be contiguous. Blank lines will be inserted in the output TP program
 - An N-code indicates a new line. In other words, one NC program instructions can spread over several lines (in NC program) until the next N-code appears.
 - A comment line also needs to start with an N-code, except for one comment right after the O-code, which becomes the program comment.
6. The position data in the NC program must be in XYZWPR (or XYZWPRE for a 7axis robot) representation, with a config data, as is used in the robot programming. The format of config string is (N L U T,0,0,0). Please note a space between letters and commas are required within the config data. Refer to the "Positional Information" section of your application-specific *Setup and Operations Manual* for the explanation of the position representation.
 7. The conversion software does not check the validity of the data. Limit checking or singularity checking should be performed by the off-line programming system when the NC program is created, or a run-time error will be posted. If an invalid numeral is specified in the positional value, an ASCII-to-Binary conversion error will be posted.
 8. Part of the positional data can be omitted. The omitted portion of the position data will be copied from the one previous motion instruction. The omission of position data is not allowed in the first motion instruction.
 9. A "motion type" of the robot motions can be specified in the source program. Because all G-codes are already defined, special M-codes are used for this purpose (M210 for Linear motion, and M211 for Joint motion). These M-codes are both "Modal."
 10. Converting TP program back to NC program is not supported.
 11. Circular motion is not supported.
 12. M-code instructions are implemented as a sub-program "Mnn," with optional numerical parameters specified by the T-code. The sub-program contents are to be provided and to be tailored by the manufacturer of the target robot system, as they are application dependent. M-codes with multiple parameters are supported up to three parameters.
 13. An M-code can be specified as a stand-alone instruction, or as part of a motion instruction, in which case the instruction is executed at the timing "TB (time-before) 0.0sec" as implemented in the robot motion sub-system.
 14. Only one M-code can be specified within one line of a program.
 15. Internally, the NC program is first converted to an ASCII format TP program, and it is then converted to binary format of a TP program. The temporary ASCII TP program is created in the internal RAM drive (RD). The maximum size of a program that can be converted is limited to the size of the RAM drive (default: 2MB). The RAM drive size can be expanded to accommodate a large program.
 16. For the convenience of the operator as a debugging aid, the intermediate TP program (in ASCII format) may be output by adding the following line as the first line of the program.

(OUTDEV XX)

where XX stands for the drive letters to represent the output device such as MC or FR.

1.3 Software Options required

In order for this feature to work correctly, the following software options are required.

1. J503 (tool offset) option is required if OFFSET/TOOL_OFFSET are to be used
2. TPP Condition Handler option is required if M-code is used within a motion command

2

OPERATION

2 OPERATION

2.1 Conversion Operation

There are two ways to use the conversion feature.

The FILE menu allows you to *load* an NC program, converting it to a TP program. If you perform a load operation on an NC program file, it will convert the file into a TP program.

Another way to start conversion is to copy the NC program file into the MD device. The MD device is a *virtual* device, and it will automatically convert the NC program file into a TP program file.

2.1.1 Load NC program in FILE menu

Performing a LOAD operation within the FILE menu will start conversion of the NC program into a TP program. See Procedure 2-1 for the operation.

Procedure 2-1 Load Operation of NC program

Conditions

- An NC program file is listed on the FILE menu screen.

Step

- 1 In the FILE menu, display the contents of the device that contains the NC programs.

MC:*.*		1/63
1 00001	NC	179
2 00002	NC	62
3 00003	NC	207
4 00004	NC	59
5 00005	NC	44
6 00006	NC	90
7 00007	NC	83
8 00008	NC	75
9 00009	NC	89
10 00010	NC	136
11 00011	NC	75

- 2 Set the cursor to the file you want to load, using the up or the down arrow keys.
- 3 Press F3 (LOAD) key, followed by F4 (YES).
- 4 The system will start loading the selected NC program. After the loading operation is over, you can verify that the converted TP program exists in the program SELECT screen.

2.1.2 COPY NC program to MD drive

Performing the file Copy operation of an NC program to the MD drive will cause the system to convert the NC program into a TP program. The COPY operation can be performed either on the FILE menu, KCL command, or via the network using the FTP (File Transfer Protocol) operation.

Please note that the MD drive is a special drive that automatically performs the conversion operation. The source NC program itself will not be found in the MD drive after the copy operation.

A

NC PROGRAM SYNTAX

A NC PROGRAM SYNTAX

Due to the special nature of using Robot as the target machine, there are some modifications/additions to the standard NC programming language.

Please also note that the following describes the default setting of the system, as the syntax can be configurable.

A.1 O code – Program Name/Number

All programs must start the O-code to specify the program number.

Syntax:

Onnn

Explanation:

1. It must start with a letter O, followed by numbers. (up to 9999)
2. This program name (Onnnn) must match the file name.
3. A comment line (without N-code) immediately following O-code becomes the “program comment”. Maximum 16 characters.

Example:

```
%06781  
(first line comment becomes the program comment)
```

A.2 N code – Line numbers

Syntax:

Nnnn

Explanation:

At the start of the block in a program an 8 digit sequence number may be specified.

1. The line number is specified after “N” letter
2. The N-code must be the first instruction within each program line.
3. The numbers become the line number in the output TP program.
4. The number may skip. However, the blank lines are inserted in the robot TP program for those skipped lines. Therefore, it may be a good idea to use contiguous line numbers to avoid number of blank lines.

Example:

N001
N7975

A.3 G code – Motion modifiers**Syntax:**

Gnnn

Explanation:

Only the following G-codes are supported in this implementation. G-code is usually followed by X, Y, Z, etc., because it is usually attached to a motion instruction. G-code is used to modify the motion instruction.

Code	Description	Modal	Explanation
G00	Rapid traverse mode	Yes	The machine moves at the fastest motion. The machine might not move on a straight line. It is converted to J (Joint) motype in the output TP program.
G01	Linear Interpolation	No	The machine moves on a straight line. This is the default setting. It is converted to L (Linear) motype in the output TP program.
G09	Exact Stop Check	No	Same as FINE term type on robot for the current motion only. See also G61.
G20	Positions in inches	Yes	Positional values are in inches in the source program (the output TP program still contains positions in millimeters) Note: When the unit is changed, all positional values have to be specified in the next motion, or uninitialized position error will be posted.
G21	Positions in millimeters	Yes	Positional values are in millimeters. Note: When the unit is changed, the all positional values have to be specified in the next motion, or uninitialized position error will be posted.
G61	Exact Stop Check (Modal)	Yes	Same as FINE term type on robot. Unlike G09, the effect remains until it is cancelled by G64.
G64	Cancels G61	Yes	This cancels the effect of G61, and set the following motions to regular “Continuous” term type. It is converted to CNT100 termtype in the output TP program.
G90	Absolute programming	Yes	This is the default setting. The positional value is absolute. When this setting is changed, the all positional values have to be specified in the next motion, or uninitialized position error will be posted.

G91	Incremental programming	Yes	The positional value is in reference to the previous position (translational moves only). When the unit is changed, the all positional values have to be specified in the next motion, or uninitialized position error will be posted.
-----	-------------------------	-----	--

A.4 F code – Feed Rate / Speed

F-code is to specify the speed of the robot.

Syntax:

Fnnn

Explanation:

For Linear motions (M210):

1. When G21 (positions in millimeters) is effective: The unit is in mm/min. In the TP program, the unit is converted to cm/min (the value is rounded to nearest integer).
2. When G20 (positions in inches) is effective: The unit is in inch/min. In the TP program, the unit is in inch/min (no conversion of the value).

For Joint motions (M211): The unit is in % (percent). In the TP program, the same speed value is used with no conversion.

If the F-code is not specified in the line, the last specified value will be used (modal setting). If it is the first line of a program, the default value specified in the system variable is used.

A.5 Position data

Syntax:

Xnnn	- value of 1st component of a position (X value)
Ynnn	- value of 2nd component of a position (Y value)
Znnn	- value of 3rd component of a position (Z value)
Wnnn	- value of 4th component of a position (W value)
Pnnn	- value of 5th component of a position (P value)
Rnnn	- value of 6th component of a position (R value)
Hnnn	- value of J7 axis (if applicable)
Innn	- value of J8 axis (if applicable)
Jnnn	- value of J9 axis (if applicable)

Explanation:

1. Refer to the "Positional Information" section of your application-specific *Setup and Operations Manual* for the explanation of the position representation.

2. All the positional data have to be specified in the first motion of a program, or “Uninitialized position” error will be posted.
3. From the second motion and after, some position components can be omitted. The previous value will be used for the omitted components.

A.6 K code - Config string

This Config information is part of the position data, and is to be specified after the XYZWPR data when position configuration needs to change.

Syntax:

K(string)

Explanation:

1. The config string value is specified as a string within the parentheses. Refer to the application-specific *Setup and Operations Manual* for the meaning of the config string.
2. If K code is omitted, the last value that appeared in the program, or the system default value will be used.
3. The format of config string is, for example, (N L U T,0,0,0). Please note that the K code string is included after the XYZWPR position data and a space between letters and commas is required (otherwise an “uninitialized position” error occurs).

Example:

N8 X0.Y-5.168Z0.410W37P0.003R0.H0I0.F780.562 K(N L U T,0,0,0)

A.7 M code – Miscellaneous codes

M-codes are generally executed by calling a TP sub-program. Because the M-codes can be defined by the user depending on the machine tool being used, the TP sub-program is to be provided and customized by the Machine Tool Builder or the user. Some of the predefined M-codes have special meaning, and are converted to equivalent TP instructions.

Syntax:

Mnn [Tmm]

Explanation:

The letter M is always followed by numbers, and by optional T-code with numbers.

Run-time processing:

1. For some pre-defined M-codes, the equivalent TP instruction will be generated.
2. Other M-codes will be converted to a CALL instruction to a specific TP sub-program.

For example, an M-code (for example, “MppTqq”, where pp and qq represent numbers) will generate an instruction to call a TP program, “Mpp(qq)” at run-time.

In this example, the TP program “Mpp” has the program name with M-code number as part of it. The T-code number (qq) is passed as the argument for the program.

3. Since the M-codes are to provide special functionality for a specific system, it is the Machine Tool Builder’s responsibility to provide the TP program “Mpp” for the M-codes used for a specific system, except for the pre-defined M-codes.
4. The number portions of the T-code are passed to the TP sub-program as the argument. If no T-code is specified, no argument will be passed.

Notes:

- You can only specify up to one M-code and up to three T-code within one line (block) of the program instruction.

A.7.1 Pre-defined M-codes

The following M-codes are pre-defined, and therefore the user may not change its meaning.

Code	Description	Modal	Explanation
M00	Compulsory stop	No	It is converted to PAUSE TP instruction.
M01	Optional stop	No	Machine will only stop at M01 if operator has pushed the optional stop button. (generates “CALL M01” instruction)
M02	End of program	No	It is converted to ABORT TP instruction.
M30	End of program with return to program top	No	It is converted to END TP instruction.

A.7.2 Special M-codes

There are some M-codes that are specific to the robot application, or require special processing during the program conversion. Please note the M-code number may be different depending on the system configuration, and therefore, these M-code numbers are configurable (by the system variable setting).

Code	Description	Modal	Explanation
M31	Vision Resync	Yes	Vision Resync command. Besides calling a sub-program, it causes subsequent motions to have a vision “offset” clause attached. (see note below)
M35	Cancel Resync offsets	Yes	Clears vision resync offset. Besides calling a sub-program, it causes the vision “offset” clauses to be removed from the subsequent motions.
M39, M60, M61	Sensor resync	Yes	Sensor Resync command. Besides calling a sub-program, it causes subsequent motions to have a sensor “offset” clause attached. (see note below)
M26	Cancel Sensor offset	Yes	Clears sensor resync offset. Besides calling a sub-program, it causes the sensor “offset” clauses to be removed from the subsequent motions.
M221	User Frame Selection	Yes	Set the user frame number to the one specified by

			the T-code
M222	User Tool selection	Yes	Set the user tool number to the one specified by the T-code
M210	Linear Robot Moves	Yes	Subsequent motions move in Linear mo-type. This instruction can be specified as a standalone instruction or as part of a motion instruction. If standalone, it generates a NOP (comment line).
M211	Joint Robot Moves	Yes	Subsequent motions move in Joint mo-type. This instruction can be specified as a standalone instruction or as part of a motion instruction. If standalone, it generates a NOP (comment line).

Note: "Offset" clause could be either OFFSET, VOFFSET, TOOL_OFFSET, depending on the configuration specified.

Other M-codes may be defined by the Machine Tool Builder or the user, by creating the corresponding TP sub-programs.

A.7.3 Predefined M-codes

The default conversion for the pre-defined M-codes are as follows.

M00 (compulsory stop)

The M-code "M00" will be converted to the PAUSE command in TP program.

Example: TP instruction for "M00" M-code

PAUSE

The functionality of PAUSE instruction is to pause the program at this line. Resuming the program will resume from the next line.

M01 (optional stop)

The M-code "M01" will be converted to a CALL instruction to the predefined sub-program "M01"

Example: TP instruction for "M01" M-code

CALL M01

The equivalent functionality of M01 does not exist on the robot, and therefore, the logic has to be created as the sub-program "M01". The TP program M01 should be provided for each application by the machine tool builder.

M02 (End of Program)

The M-code "M02" will be converted to the ABORT command in TP program.

Example: TP instruction for “M02” M-code

ABORT

The functionality of ABORT instruction is to abort the program at this line. Once aborted, the program may not be resumed. (You may “start” the program from the current line, however).

M30 (End of program)

The M-code “M30” will be converted to the END command in TP program.

Example: TP instruction for “M30” M-code

END

The functionality of END instruction is to finish the program at this line, and set the cursor back to the beginning (if it is the main program). If the program is executed as a sub-program, END instruction works as the RETURN instruction to go back to the caller program and continue. Please note the functionality of END statement may not be exactly equivalent to M30 code in the typical NC machines.

M221 (User Frame Selection)**Syntax:**

M221Tnn - Selection of User Frame number

Explanation:

It generates “UFRAME_NUM=nn” instruction.

It is not a standard M-code definition. This is a new M-code specifically for robot, due to the requirement for the way how positions are specified on robots. The functionality of M221 is as follows.

- For program convertor – This will set the user frame number for all the positions thereafter (user frame number is part of the position data)
- For program execution – This will generate a code to set the user frame number at run-time

Example: TP instruction for “M221” M-code

uframe_num = nn

The frame number should be specified by a T-code that follows.

M222 (User Tool Selection)**Syntax:**

M222Tnn - Selection of User Frame number

Explanation:

It generates “UTOOL_NUM=nn” instruction.

It is not a standard M-code definition. This is a new M-code specifically for robot, due to the requirement for the way how positions are specified on robots. The number can be configurable. The functionality of M222 is as follows.

- For program convertor – This will set the user tool number for all the positions thereafter (user tool number is part of the position data)
- For program execution – This will generate a code to set the user tool number for run-time

Example: TP instruction for “M222” M-code

```
utool_num = nn
```

The TOOL number should be specified by a T-code that follows.

A.7.4 Other (user defined) M-codes***M-codes with no associated T-code***

Because such an M-code has no T-code associated, the TP sub-program will be called with no argument.

Example: TP instruction for “M25” M-code

```
CALL M25
```

Because the functionality of M25 is machine dependent, the TP program M25 will have to be provided by the Machine Tool Builder. In many cases, the program will communicate with the peripheral equipment via process I/O interface.

Please note the name of the sub-program will be composed of the command letter “M” followed by the command number letters exactly as specified.

M-codes with associated T-code

The TP sub-program for such an M-code will be called with the argument that is specified as T-code. Up to three argument may be specified.

Example: TP instruction for “M56T4321” M-code

```
CALL M56(4321)
```

The meaning of M-code “M56” is machine dependent, and therefore, the TP program M56 will have to be defined by the Machine Tool Builder. In most cases, the program will communicate with the peripheral equipment via process I/O interface.

B SYSTEM VARIABLES

B SYSTEM VARIABLES

B.1 \$NC_SYTX_TBL

This variable is to define command letters of NC language with their meaning.

- \$NC_SYTX_TBL[].\$nc_inst_str: STRING - To hold the command letter for the NC instruction
- \$NC_SYTX_TBL[].\$nc_inst_sfx : STRING - Optional suffix character for NC instruction
- \$NC_SYTX_TBL[].\$nc_inst_typ : integer - Instruction type for the NC instruction
- \$NC_SYTX_TBL[].\$nc_elem_idx : integer - To specify the field index within the pos data structure. For example, X is the first index (0), Y being the next (index 1), etc.

The meaning of inst_type is as follows.

- 1: O-code
- 2: N-code
- 3: comment line
- 4: M-code, further classified in M-code syntax table
- 5: G-code, further classified in G-code syntax table
- 6: T-code, as an argument for M-code
- 7: F-code, feed rate
- 8: Config string (K-code)
- 9: position field (X, Y, Z, W, P, R or like)

For example, the following will define the command “Y” to indicate the position value for the 2nd positional field.

- \$NC_SYTX_TBL[1].\$nc_inst_str = “Y”
- \$NC_SYTX_TBL[1].\$nc_inst_sfx = “”
- \$NC_SYTX_TBL[1].\$nc_inst_typ = 9
- \$NC_SYTX_TBL[1].\$nc_elem_idx = 1

B.2 \$NC_GCOD_TBL : To define G-code numbers with their meaning.

- \$NC_GCOD_TBL[].\$nc_gvalue : STRING – To hold the numerical value for a G-code
- \$NC_GCOD_TBL[].\$nc_gtype : INTEGER - Code type for this G-code
- \$NC_GCOD_TBL[].\$nc_gmodal : BOOLEAN - Whether this G-code is modal or not

The meaning of Gtype is as follows.

- 1: Rapid traverse motion (default: G00)
- 2: Linear interpolation (default: :G01)
- 3: Exact stop check (default: G09)
- 4: Positions in inches (default: G20)
- 5: Positions in millimeters (default: G21)
- 6: Exact stop check (modal) (default: G61)
- 7: Cancels FINEMODE (default: G64)
- 8: Absolute programming (default: G90)

- 9: Incremental programming (default: G91)
- 10: Reposition origin point (default: G92)
- 11: Preset origin point (default: G92.1)

For example, the following defines the G09 code.

- `$NC_GCOD_TBL[1].$nc_gvalue = "9"`
- `$NC_GCOD_TBL[1].$nc_gtype = 3`
- `$NC_GCOD_TBL[1].$nc_gmodal = FALSE`

B.3 `$NC_MCOD_TBL` : To define M-code numbers with their meaning.

- `$nc_mvalue` : STRING - Numerical value for an M-code
- `$nc_mtype` : INTEGER - Instruction type for this M-code
- `$nc_call_sub` : BOOLEAN - Whether sub-prog is called
- `$nc_ofst_typ` : specifies the type of offset (bit mask)
 - 1: OFFSET
 - 2: TOOL_OFFSET
 - 4: VOFFSET
- `$nc_mmodal` : BOOLEAN - Whether this M-code is modal or not

The meaning of `mtype` is as follows.

- 1: PAUSE : for M00
- 2: ABORT : for M02
- 3: END : for M30
- 4: UFRAME : for M221
- 5: UTOOL : for M222
- 6: LIN : Enable Linear motion M210
- 7: JNT : Enable Joint motion M211
- 8: OFST : Enable offset (type of offset specified by `$offset_type`)
- 9: CLR : Clear offset

For example, the following will define M31 (Vision resync) command:

- `$NC_MCOD_TBL[1].$nc_mvalue = "31"`
- `$NC_MCOD_TBL[1].$nc_mtype = 8`
- `$NC_MCOD_TBL[1].$nc_call_sub = TRUE`
- `$NC_MCOD_TBL[1].$nc_ofst_typ = 1 (offset)`
- `$NC_MCOD_TBL[1].$nc_mmodal = TRUE`

B.4 `$NC_DEFMODAL`

This variable is to define the default value of the modal values, and is used every time a new NC program is being read.

The following is the default value of this variable.

- `$NC_DEFMODAL.$INST.$posid = 0`
- `$NC_DEFMODAL.$INST.$motype = 1 (Linear)`
- `$NC_DEFMODAL.$INST.$termtype = 1 (CNT)`

- \$NC_DEFMODAL.\$INST.\$stempct = 100 (100%)
- \$NC_DEFMODAL.\$INST.\$spdunit = 0 (mm/min)
- \$NC_DEFMODAL.\$INST.\$lin_speed = 6000 (6000 mm/min = 100 mm/sec)
- \$NC_DEFMODAL.\$INST.\$jnt_speed = 100 (100%)
- \$NC_DEFMODAL.\$INST.\$incremental = 0 (false)
- \$NC_DEFMODAL.\$INST.\$use_offset = 0 (bit mask)
- \$NC_DEFMODAL.\$POS.\$posid = 0
- \$NC_DEFMODAL.\$POS.\$sframenum = 0(\$sframe_num)
- \$NC_DEFMODAL.\$POS.\$sutilnum = 1 (\$sutil_num)
- \$NC_DEFMODAL.\$POS.\$numposelem = 6 (number of pos data elements)
- \$NC_DEFMODAL.\$POS.\$posrep = 0 (XYZWPR)
- \$NC_DEFMODAL.\$POS.\$posunit = 0 (mm)
- \$NC_DEFMODAL.\$POS.\$comment = ""
- \$NC_DEFMODAL.\$POS.\$config = "N L U T, 0, 0, 0 "
- \$NC_DEFMODAL.\$POS.\$pos1_value = uninit
- \$NC_DEFMODAL.\$POS.\$pos2_value = uninit
- \$NC_DEFMODAL.\$POS.\$pos3_value = uninit
- \$NC_DEFMODAL.\$POS.\$pos4_value = uninit
- \$NC_DEFMODAL.\$POS.\$pos5_value = uninit
- \$NC_DEFMODAL.\$POS.\$pos6_value = uninit
- \$NC_DEFMODAL.\$POS.\$pos7_value = uninit
- \$NC_DEFMODAL.\$POS.\$pos8_value = uninit
- \$NC_DEFMODAL.\$POS.\$pos9_value = uninit
- \$NC_DEFMODAL.\$ATTR.\$protect = 1
- \$NC_DEFMODAL.\$ATTR.\$stacksize = 0
- \$NC_DEFMODAL.\$ATTR.\$priority = 50
- \$NC_DEFMODAL.\$ATTR.\$timeslice = 0
- \$NC_DEFMODAL.\$ATTR.\$busylampoff = 0
- \$NC_DEFMODAL.\$ATTR.\$abortreq = 0
- \$NC_DEFMODAL.\$ATTR.\$pausereq = 0
- \$NC_DEFMODAL.\$ATTR.\$defaultgrp = 1
- \$NC_DEFMODAL.\$ATTR.\$controlcode = 0

B.5 \$NC_OFST_CMD

This variable indicates the command string to be used for Offset command (when the mask 1 is set in \$NC_MCOD_TBL[.].\$nc_ofst_tpy system variable)

Default value: "OFFSET"

For example, when the vision sensor is to be used, the motion instruction will add the specified Offset clause to its statements after a vision resync command (e.g. M31) is executed until a vision clear command (e.g. M35) is executed. The user can specify the actual offset clause in a system variable (\$NC_OFST_CMD) as a string data (The conversion software just copies the string as the "offset" clause).

B.6 \$NC_TOFS_CMD

This variable indicates the command string to be used for Tool_Offset command (when the mask 2 is set in \$NC_MCOD_TBL[].\$nc_ofst_typ system variable)

Default value: "TOOL_OFFSET"

For example, if the proximity sensor is enabled, the motion instruction will add the specified TOOL_OFFSET clause to its statements after a sensor resync command (e.g. M39/M60/M61) is executed until a sensor clear command (e.g. M26) is executed. The user can specify the actual tool_offset clause in a system variable (\$NC_TOFST_CMD) as a string data (The conversion software just copies the string as the "tool_offset" clause).

B.7 \$NC_VOFS_CMD : Command string to be used for Vision offset command

This variable indicates the command string to be used for Vision Offset command (when the mask 4 is set in \$NC_MCOD_TBL[].\$nc_ofst_typ system variable)

Default value: uninitialized

B.8 \$NC_DEST_DRV

The device name for the intermediate file. This value should be always "MD"



NC PROGRAM CONVERSION EXAMPLE

C NC PROGRAM CONVERSION EXAMPLE

Input G-code Program:

```
%06781
(first line comment becomes the program comment)
N1 (,140A4112-678,2GNCD,BANNON,REV003,1-1,FWC,1G2K,1G2K,CAT)
N2 (MCD,REV003...SYSTEM.034)
N3 (UNIX.POST.GEM5F2,OPTION.5...,REV.16..)
N4M52T3
N5M00(LOAD.RIVET.OFFSET)
(6/32.OFFSET.RIVET.ANVIL.3)
(6/32.RIVET)
N6M26
N7M211X130.Y123.4Z81.W81.P0.R180.F30.
N8M210X130.Y123.4Z0.W0.P0.R180.F300.
N9M40T0
N10M39
N11M31T2
N12M60
N13X130.Y-33.Z-47.12W-47.12P-70.R350.
N14X130.Y8.Z-35.12W-35.12P0.R270.F300.
N15X95.228Y7.152Z-55.677W-2.538P1.475R270.M80T0355F230.
(ANVIL:265)
N16X95.228Y7.152Z-57.677W-4.538W1.475R270.
N17M35
N18M26
N19M211X130.Y123.4Z81.W81.P0.R180.F30.
N20M56T60
```

Output TP program (in ASCII format):

```
File Name: 06781.LS

/PROG 06781
/ATTR
COMMENT = "first line comment becomes the program comment";
PROTECT          = READ_WRITE;
TCD:  STACK_SIZE = 0,
      TASK_PRIORITY = 50,
      TIME_SLICE = 0,
      BUSY_LAMP_OFF = 0,
      ABORT_REQUEST = 0,
      PAUSE_REQUEST = 0;
DEFAULT_GROUP    = 1,*,*,*,*;
CONTROL_CODE     = 00000000 00000000;
/MN
1: ! ,140A4112-678,2GNCD,BANNON,REV003,1-1,FWC,1G2K,1G2K,CAT ;
2: ! MCD,REV003...SYSTEM.034 ;
```

```

3: ! UNIX.POST.GEM5F2,OPTION.5...,REV.16.. ;
4: CALL M52(3) ;
5: PAUSE ;
6: CALL M26 ;
7: J P[7] 30% FINE ;
8: L P[8] 300mm/sec FINE ;
9: CALL M40(0) ;
10: CALL M39 ;
11: CALL M31(2) ;
12: CALL M60 ;
13: L P[13] 300mm/sec FINE TOOL_OFFSET ;
14: L P[14] 300mm/sec FINE TOOL_OFFSET ;
15: L P[15] 230mm/sec FINE TB 0.00sec,CALL M80(0355) TOOL_OFFSET ;
16: L P[16] 230mm/sec FINE TOOL_OFFSET ;
17: CALL M35 ;
18: CALL M26 ;
19: J P[19] 30% FINE ;
20: CALL M56(60) ;

```

/POS

```

P[7]{
  GP1:
    UF : 0, UT : 1,          CONFIG : 'N U T, 0, 0, 0',
    X = 130.000 mm, Y = 123.400 mm, Z = 81.000 mm,
    W = 81.000 deg, P = 0.000 deg, R = 0.000 deg
};
P[8]{
  GP1:
    UF : 0, UT : 1,          CONFIG : 'N U T, 0, 0, 0',
    X = 130.000 mm, Y = 123.400 mm, Z = 0.000 mm,
    W = 0.000 deg, P = 0.000 deg, R = 180.000 deg
};
P[13]{
  GP1:
    UF : 0, UT : 1,          CONFIG : 'N U T, 0, 0, 0',
    X = 130.000 mm, Y = -33.000 mm, Z = -47.120 mm,
    W = -47.120 deg, P = -70.000 deg, R = 350.000 deg
};
P[14]{
  GP1:
    UF : 0, UT : 1,          CONFIG : 'N U T, 0, 0, 0',
    X = 130.000 mm, Y = 8.000 mm, Z = -35.120 mm,
    W = -35.120 deg, P = 0.000 deg, R = 270.000 deg
};
P[15]{
  GP1:
    UF : 0, UT : 1,          CONFIG : 'N U T, 0, 0, 0',
    X = 95.228 mm, Y = 7.152 mm, Z = -55.677 mm,
    W = -2.538 deg, P = 1.475 deg, R = 270.000 deg
};
P[16]{
  GP1:

```

```
      UF : 0, UT : 1,          CONFIG : 'N U T, 0, 0, 0',
      X =  95.228  mm, Y =      7.152  mm,    Z = -57.677  mm,
      W =  -4.538 deg, P =      1.475 deg,    R =  270.000 deg
};
P[19]{
  GP1:
    UF : 0, UT : 1,          CONFIG : 'N U T, 0, 0, 0',
    X = 130.000  mm, Y =      123.400  mm,    Z =  81.000  mm,
    W =  81.000 deg, P =      0.000 deg, R =      0.000 deg
};
/END
```